

Optimization for Data Science

Project – Robust Regression

Ali LARIANE

M2 MIAE ID – 2025/2026

1 Introduction

Dans ce projet, nous étudions un problème de régression robuste basé sur la fonction de perte biweight :

$$\phi(t) = \frac{t^2}{1+t^2}.$$

Cette fonction est quadratique près de zéro mais saturante pour les grandes valeurs, ce qui permet de réduire l'influence des outliers. Cependant, cette perte est non convexe, ce qui rend l'optimisation plus difficile.

Nous étudions successivement :

- le gradient descent sur un problème non convexe,
- le stochastic gradient,
- l'effet de la régularisation ℓ_2 et du proximal gradient.

2 Question 1 – Toy problem : non-nullité de l'optimum et point selle

On considère le jeu de données jouet

$$X = \begin{pmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad y = \begin{pmatrix} 1 \\ -1 \\ 1 \\ 1 \end{pmatrix},$$

et la fonction objectif

$$f(w) = \frac{1}{2n} \sum_{i=1}^n \phi(x_i^\top w - y_i), \quad \phi(t) = \frac{t^2}{1+t^2}, \quad n = 4.$$

On rappelle que $\phi(t) \geq 0$ pour tout t , et que $\phi(t) = 0$ si et seulement si $t = 0$.

(a) Justifier que la valeur optimale ne peut pas être 0

Supposons par l'absurde qu'il existe $w \in \mathbb{R}^2$ tel que $f(w) = 0$. Comme f est une moyenne de termes positifs, cela implique nécessairement

$$\phi(x_i^\top w - y_i) = 0 \quad \forall i \quad \Longleftrightarrow \quad x_i^\top w - y_i = 0 \quad \forall i.$$

Autrement dit, il faudrait que le système linéaire $Xw = y$ soit satisfiable. Écrivons les équations :

- $(-1, 0) \cdot (w_1, w_2) = 1 \implies -w_1 = 1 \implies w_1 = -1,$
- $(0, -1) \cdot (w_1, w_2) = -1 \implies -w_2 = -1 \implies w_2 = 1,$

- $(1, 0) \cdot (w_1, w_2) = 1 \implies w_1 = 1,$
- $(0, 1) \cdot (w_1, w_2) = 1 \implies w_2 = 1.$

On obtient une contradiction puisque les deux équations sur w_1 imposent simultanément $w_1 = -1$ et $w_1 = 1$. Donc le système $Xw = y$ n'a pas de solution, et il est impossible d'annuler tous les résidus.

Par conséquent, il n'existe aucun w tel que $f(w) = 0$ et la valeur optimale vérifie

$$\min_w f(w) > 0.$$

(b) Montrer que $\bar{w} = (0, 1)^\top$ est un point selle

On pose

$$\bar{w} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Commençons par calculer les résidus $r_i(\bar{w}) = x_i^\top \bar{w} - y_i$:

$$X\bar{w} = \begin{pmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \\ 0 \\ 1 \end{pmatrix}, \quad r(\bar{w}) = X\bar{w} - y = \begin{pmatrix} -1 \\ 0 \\ -1 \\ 0 \end{pmatrix}.$$

Le gradient de f s'écrit (formule du sujet)

$$\nabla f(w) = \frac{1}{n} X^\top z(w), \quad z_i(w) = \frac{r_i(w)}{(1 + r_i(w)^2)^2}.$$

À $w = \bar{w}$, on a $r_1 = r_3 = -1$ et $r_2 = r_4 = 0$, donc

$$z(\bar{w}) = \begin{pmatrix} -\frac{1}{(1+1)^2} \\ 0 \\ -\frac{1}{(1+1)^2} \\ 0 \end{pmatrix} = \begin{pmatrix} -\frac{1}{4} \\ 0 \\ -\frac{1}{4} \\ 0 \end{pmatrix}.$$

Ainsi,

$$X^\top z(\bar{w}) = \begin{pmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} -\frac{1}{4} \\ 0 \\ -\frac{1}{4} \\ 0 \end{pmatrix} = \begin{pmatrix} (-1)(-\frac{1}{4}) + (1)(-\frac{1}{4}) \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Donc

$$\nabla f(\bar{w}) = \frac{1}{n} X^\top z(\bar{w}) = 0,$$

ce qui montre que $\bar{w}$ est un *point critique*.

Pour conclure que $\bar{w}$ est un point selle, il faut vérifier que $\bar{w}$ est un point critique mais qu'il n'est pas un minimum local. En effet, on a $\nabla f(\bar{w}) = 0$, donc $\bar{w}$ est bien un point critique.

Cependant, on observe sur la carte de niveaux qu'il existe des points arbitrairement proches de $\bar{w}$ (par exemple autour de $(\pm 1, 1)$) pour lesquels la valeur de l'objectif est plus petite. Ainsi, $\bar{w}$ ne peut pas être un minimum local.

De plus, on constate qu'il existe des directions dans lesquelles la fonction diminue (direction horizontale le long de w_1) et d'autres directions dans lesquelles elle augmente (direction verticale le long de w_2). Par définition, un point critique qui possède à la fois des directions de montée et de descente est un point selle. Ainsi, $\bar{w}$ est bien un point selle de la fonction f .

3 Question 2 – Analyse de Gradient Descent sur le toy problem

Dans cette question, on analyse le comportement de la descente de gradient (GD) sur le toy problem, en particulier autour du point selle

$$\bar{w} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Les figures du notebook (carte de niveaux et trajectoires) montrent deux minima (situés de part et d'autre de l'axe $w_1 = 0$) ainsi qu'un point critique au centre, identifié comme point selle à la Question 1.

(a) Pourquoi la première exécution (depuis $w_0 = (0, 1.4)^\top$) converge vers $\bar{w}$?

On considère une descente de gradient de la forme

$$w_{k+1} = w_k - \alpha \nabla f(w_k),$$

avec un pas $\alpha > 0$.

L'observation clé est que le point initial $w_0 = (0, 1.4)^\top$ est choisi *exactement* sur l'axe $w_1 = 0$. Or, pour ce toy problem, la fonction f possède une symétrie gauche/droite (liée au fait que les lignes $x_1 = (-1, 0)$ et $x_3 = (1, 0)$ jouent des rôles symétriques et ont la même cible $y_1 = y_3 = 1$). Cette symétrie implique que, lorsque $w_1 = 0$, les contributions au gradient selon la direction w_1 se compensent :

$$(\nabla f(w))_1 = 0 \quad \text{lorsque } w_1 = 0.$$

Par conséquent, si l'on démarre avec $w_{0,1} = 0$, alors l'itération de GD conserve (idéalement) cette propriété et la suite (w_k) reste sur la droite $w_1 = 0$ (à des erreurs numériques près).

Une fois contraint sur cette droite, GD ne peut évoluer que selon w_2 . La dynamique le long de w_2 conduit alors naturellement vers le point critique situé sur cet axe, qui est précisément $\bar{w} = (0, 1)^\top$. Ainsi, l'algorithme converge vers $\bar{w}$ non pas parce que c'est un minimum, mais parce que :

- le point de départ appartient à une trajectoire symétrique particulière,
- sur cette trajectoire, GD ne peut pas “partir” vers les minima situés près de $(\pm 1, 1)$,
- il atteint un point critique (gradient nul) et s'arrête.

Cette situation correspond à ce qui est observé dans le notebook : la trajectoire issue de $w_0 = (0, 1.4)^\top$ finit au point rouge $\bar{w}$, alors que ce point n'est pas un minimum.

(b) Comment l'expérience confirme-t-elle le résultat théorique (2015) : GD échappe aux points selles pour presque tout point initial ?

Un résultat classique en optimisation non convexe (mentionné dans le sujet) est que, sous des hypothèses de régularité, la descente de gradient avec un pas convenable *échappe* aux points selles pour *presque tout* point initial. Plus précisément, l'ensemble des points initiaux conduisant exactement à un point selle est “très petit” (de mesure nulle) : il correspond en pratique à des configurations particulières, souvent liées à des symétries ou à des variétés stables de dimension plus faible.

L'expérience numérique confirme ce phénomène :

- Lorsque l'on choisit volontairement un point initial *spécial* (ici w_0 avec $w_{0,1} = 0$), GD peut converger vers le point selle.
- En revanche, lorsque l'on répète l'expérience avec des points initiaux *aléatoires* (donc presque sûrement en dehors de cet ensemble spécial), GD *n'atteint pas* le point selle et converge vers l'un des deux minima locaux.

Ceci est exactement ce que l'on observe dans le notebook : sur un grand nombre d'essais aléatoires, GD échappe systématiquement au point selle et descend vers une des deux cuvettes. L'expérience illustre donc que la convergence vers un point selle est un cas particulier (rare), tandis que l'échappement est le comportement typique en pratique.

4 Question 3 – Analyse de Gradient Descent sur les données synthétiques

Dans cette partie, on applique la descente de gradient à la régression robuste sur un jeu de données synthétique plus grand. On compare deux stratégies de choix du pas :

- **pas constant** (pas fixé à l'avance),
- **recherche linéaire d'Armijo** (pas adaptatif).

On analyse les courbes fournies par le notebook : évolution de l'objectif $f(w_k)$ et de la norme du gradient $\|\nabla f(w_k)\|$.

(a) Quelle stratégie de pas semble la plus efficace ? (en tenant compte du coût)

Observation sur l'objectif. D'après les courbes, la stratégie *Armijo* fait décroître l'objectif plus rapidement et atteint une valeur finale plus faible que le pas constant. Le pas constant progresse plus lentement et peut sembler stagner après un certain nombre d'itérations (plateau).

Interprétation. La recherche linéaire d'Armijo choisit automatiquement un pas α_k assurant une diminution suffisante de l'objectif (condition d'Armijo). Intuitivement :

- lorsque la région est “facile”, Armijo autorise des pas plus grands et accélère la descente ;
- près des zones difficiles (vallées étroites, non-convexité), Armijo réduit le pas pour éviter des augmentations de f .

Ainsi, Armijo est souvent plus robuste et plus efficace en nombre d'itérations.

Coût des deux stratégies. Cependant, Armijo est plus coûteux par itération : une recherche linéaire teste plusieurs valeurs de pas et nécessite donc plusieurs *évaluations de la fonction objectif* f (et parfois des évaluations de gradient supplémentaires selon l'implémentation). Au contraire, avec un pas constant, chaque itération nécessite typiquement :

- 1 calcul de gradient (coût $\sim O(nd)$),
- 1 évaluation de f (si on la stocke pour tracer les courbes).

Conclusion (efficacité).

- Si l'on juge l'efficacité uniquement en *nombre d'itérations* ou en *valeur finale atteinte*, **Armijo est meilleure.**
- Si l'on tient compte du *coût en évaluations de f* (et donc du temps de calcul), le pas constant peut être compétitif car il est bien moins cher par itération.

Ainsi, le choix dépend de l'objectif pratique : *solution de meilleure qualité* (Armijo) vs *coût minimal* (pas constant).

(b) Les courbes de normes de gradient correspondent-elles au taux théorique $O(1/\sqrt{K})$? Est-ce surprenant ?

Pour une fonction non convexe à gradient Lipschitz, une garantie classique pour GD (avec un pas approprié) est :

$$\min_{0 \leq k \leq K} \|\nabla f(w_k)\| = O\left(\frac{1}{\sqrt{K}}\right).$$

Cette borne est *faible* : elle concerne le meilleur itéré jusqu'à K et non forcément $\|\nabla f(w_K)\|$, et elle est asymptotique.

Observation sur les courbes. Dans nos résultats :

- avec **pas constant**, la norme du gradient décroît globalement mais pas exactement selon une loi en $1/\sqrt{K}$ (la pente n'est pas parfaitement linéaire en échelle log-log, et on observe parfois un ralentissement) ;
- avec **Armijo**, la norme du gradient est plus irrégulière : on observe des oscillations/pics, car le pas varie et l'objectif est non convexe.

Interprétation. Ce comportement n'est **pas surprenant** pour plusieurs raisons :

1. La borne $O(1/\sqrt{K})$ est une garantie théorique pessimiste et asymptotique ; en pratique les courbes peuvent être meilleures ou plus chaotiques.
2. La fonction est **non convexe** : l'algorithme peut traverser des zones plates, des vallées, des cols, ce qui perturbe la décroissance régulière de $\|\nabla f(w_k)\|$.
3. La stratégie **Armijo** modifie le pas à chaque itération ; cela améliore souvent la descente de f mais peut produire des oscillations de la norme du gradient.

Conclusion. Les courbes ne suivent pas exactement une décroissance en $1/\sqrt{K}$, mais elles restent qualitativement cohérentes avec la théorie : la norme du gradient diminue globalement, tout en présentant des effets non idéaux dus à la non-convexité et (pour Armijo) à l'adaptation du pas.

5 Question 4 – Stochastic Gradient sur le toy problem

Dans cette partie, on considère la fonction objectif sous la forme d'une somme finie :

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w), \quad f_i(w) = \frac{1}{2} \phi(x_i^\top w - y_i),$$

où $\phi(t) = \frac{t^2}{1+t^2}$ et $n = 4$.

On étudie le comportement du stochastic gradient descent (SGD) et de ses variantes sur le toy problem.

(a) Batch $\{2, 4\}$ et équivalence avec l'indice 4

On considère le batch $S = \{2, 4\}$. Les points correspondants sont :

$$x_2 = (0, -1), \quad y_2 = -1, \quad x_4 = (0, 1), \quad y_4 = 1.$$

Les résidus associés sont :

$$r_2(w) = x_2^\top w - y_2 = -w_2 + 1, \quad r_4(w) = x_4^\top w - y_4 = w_2 - 1.$$

On remarque que :

$$r_2(w) = -r_4(w).$$

La dérivée de la fonction biweight est donnée par :

$$\psi(t) = \phi'(t) = \frac{t}{(1+t^2)^2},$$

qui est une fonction **impair**, c'est-à-dire :

$$\psi(-t) = -\psi(t).$$

Le gradient stochastique associé au batch S est :

$$g_S(w) = \frac{1}{|S|} \sum_{i \in S} \psi(r_i(w))x_i = \frac{1}{2} (\psi(r_2(w))x_2 + \psi(r_4(w))x_4).$$

En utilisant l'impairité de ψ et la relation $r_2(w) = -r_4(w)$, on obtient :

$$g_S(w) = \frac{1}{2} (-\psi(r_4(w))(0, -1) + \psi(r_4(w))(0, 1)) = \psi(r_4(w))(0, 1).$$

Ce gradient est colinéaire au gradient associé au seul indice 4. Ainsi, utiliser le batch $\{2, 4\}$ est équivalent (à un facteur de pas près) à utiliser uniquement l'exemple $i = 4$.

(b) Comparaison des taux de convergence théoriques

Pour la descente de gradient classique (GD), dans un cadre non convexe, on dispose de la garantie :

$$\min_{0 \leq k \leq K} \|\nabla f(w_k)\| = O\left(\frac{1}{\sqrt{K}}\right).$$

Pour le stochastic gradient (SGD), la variance du gradient introduit du bruit supplémentaire, et le taux théorique est plus lent :

$$\min_{0 \leq k \leq K} \mathbb{E}\|\nabla f(w_k)\| = O\left(K^{-1/4}\right).$$

Ainsi, du point de vue purement théorique, SGD converge plus lentement que GD en nombre d'itérations.

(c) SGD peut-il être meilleur que GD sur le toy problem ?

Sur le toy problem, on a $n = 4$, ce qui signifie que le coût du calcul du gradient complet est très faible. Dans ce contexte, GD est quasiment aussi peu coûteux que SGD, mais bénéficie d'une direction de descente exacte (sans bruit).

Par conséquent :

- SGD introduit une variance inutile dans les mises à jour,
- GD est plus stable et converge plus efficacement.

Ainsi, sur ce problème jouet de très petite taille, **SGD ne peut pas être meilleur que GD** en pratique. L'intérêt du stochastic gradient apparaît surtout lorsque n est très grand, ce qui n'est pas le cas ici.

6 Question 5 – Analyse des méthodes stochastiques sur la régression robuste

Dans cette question, on analyse les performances de méthodes de type stochastic gradient appliquées au problème de régression robuste. On compare plusieurs tailles de batch et on les met en regard de la descente de gradient (GD) “plein batch”. Les courbes fournies par le notebook représentent l'évolution de l'objectif (en échelle log) en fonction du nombre d'époques.

Les méthodes comparées sont :

- **SG** (batch size = 1),
- **Batch-SG** avec batch size $\approx n/200, n/60, n/20$,
- **GD** (batch size = n , une itération par époque).

Les expériences sont menées en veillant à comparer des méthodes ayant traité approximativement la même quantité de données, c'est-à-dire un même nombre d'époques.

(a) D'après le plot, SG est-il meilleur que GD ?

Observation sur le graphe. Sur la figure de convergence :

- la courbe **SG (batch=1)** décroît le plus rapidement et atteint la plus faible valeur d'objectif après 100 époques ;
- les méthodes **Batch-SG** avec batch $n/200$ puis $n/60$ décroissent également, mais moins vite que SG ;
- **GD (batch=n)** est la courbe la plus lente et apparaît presque plate à l'échelle du graphe, indiquant une progression beaucoup plus faible.

Ainsi, en termes de diminution de l'objectif au fil des époques, les méthodes stochastiques (en particulier SG) surpassent nettement GD sur cette expérience.

Interprétation. Ce comportement est cohérent avec le rôle du stochastic gradient :

- SG effectue des mises à jour très fréquentes (une mise à jour après chaque exemple), ce qui permet de progresser rapidement, en particulier en début d'optimisation ;
- GD, au contraire, n'effectue qu'une mise à jour par époque (après avoir traité tout le jeu de données), ce qui ralentit la progression initiale ;
- le bruit dû au gradient stochastique peut jouer un rôle favorable en non-convexe : il peut aider à éviter certaines zones plates ou certains points critiques non minimaux.

Conclusion. D'après le plot, **SG (et plus généralement les méthodes stochastiques) performe mieux que GD** sur cette instance, au moins sur l'horizon d'époques considéré.

(b) Quelle taille de batch est la plus efficace ? Comment proposer une meilleure valeur ?

Observation sur le graphe. On observe une hiérarchie claire :

$\text{SG (batch=1)} > \text{Batch-SG (} n/200 \text{)} > \text{Batch-SG (} n/60 \text{)} > \text{Batch-SG (} n/20 \text{)} > \text{GD (batch=n)}.$

Plus le batch est petit, plus la convergence (en termes d'objectif par époque) est rapide dans cette expérience.

Interprétation : compromis biais/variance/coût. La taille de batch contrôle un compromis classique :

- **petit batch** : gradient très bruité mais mises à jour très fréquentes, ce qui peut accélérer la descente (surtout au début) ;
- **grand batch** : gradient plus précis (variance plus faible) mais mises à jour moins fréquentes et coût par itération plus élevé, donc progrès plus lents par époque ;
- **batch intermédiaire** : compromis entre stabilité (variance réduite) et rapidité (mises à jour encore fréquentes).

Meilleure taille observée. Sur ce graphe, **batch=1** est le plus performant. Cependant, batch=1 peut être plus bruité et peut parfois conduire à des oscillations plus importantes selon les données et le choix de pas. Dans ce cas, un batch intermédiaire (ici $n/200$) reste très compétitif tout en étant plus stable.

Comment suggérer une meilleure valeur ? Une stratégie raisonnable consiste à :

- raffiner la recherche autour des meilleures valeurs observées (par exemple tester $n/150$, $n/100$, $n/80$) ;
- ou utiliser des tailles standard en pratique (32, 64, 128) si n est grand, puis sélectionner celle qui minimise l'objectif le plus vite en temps de calcul réel.

L'objectif est de trouver un batch suffisamment petit pour conserver des mises à jour fréquentes, mais suffisamment grand pour réduire la variance du gradient et stabiliser la convergence.

Conclusion. D'après la figure, les plus efficaces sont les **petits batchs** (en particulier batch=1). Pour une meilleure robustesse, il est pertinent de tester des batchs légèrement plus grands autour de $n/200$ afin de trouver le meilleur compromis vitesse/stabilité.

7 Question 6 – Effet de la régularisation et comparaison GD vs Proximal Gradient

On considère la version régularisée du problème :

$$f_2(w) = f(w) + \frac{\lambda}{2}\|w\|^2, \quad \lambda > 0,$$

où f est l'objectif non convexe associé à la perte biweight. On note $g(w) = \frac{\lambda}{2}\|w\|^2$ la partie régularisation.

(a) Justifier que le point selle de la Question 1 n'est plus un point selle de f_2 pour $\lambda > 0$

On rappelle que pour la fonction non régularisée f , le point

$$\bar{w} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

est un *point selle* : il s'agit d'un point critique ($\nabla f(\bar{w}) = 0$) mais la Hessienne $\nabla^2 f(\bar{w})$ admet des directions de courbure négative (donc $\bar{w}$ n'est pas un minimum local).

Considérons maintenant la fonction régularisée f_2 . On a :

$$\nabla f_2(w) = \nabla f(w) + \lambda w, \quad \nabla^2 f_2(w) = \nabla^2 f(w) + \lambda I.$$

Ainsi, la Hessienne de f_2 s'obtient en ajoutant λI à la Hessienne de f . Cela signifie que **toutes les valeurs propres** de la Hessienne sont décalées de λ : si μ est une valeur propre de $\nabla^2 f(\bar{w})$, alors $\mu + \lambda$ est une valeur propre de $\nabla^2 f_2(\bar{w})$.

En particulier, les directions de courbure négative de f (valeurs propres $\mu < 0$) sont “remontées” vers le positif. Pour λ suffisamment grand, on obtient $\mu + \lambda > 0$ pour toutes les valeurs propres, ce qui rend $\nabla^2 f_2(\bar{w})$ définie positive. Dans ce cas, $\bar{w}$ ne peut plus être un point selle : il devient (au moins localement) un point de courbure positive. Intuitivement, la régularisation ℓ_2 rend la géométrie plus “convexe” en pénalisant les grandes normes de w , ce qui supprime les directions instables responsables du point selle.

(b) Itérations de GD et de Proximal GD pour f_2 à partir de $w_0 = \bar{w}$

Descente de gradient (GD) sur f_2 . La descente de gradient appliquée à f_2 s'écrit :

$$w_{k+1} = w_k - \alpha \nabla f_2(w_k) = w_k - \alpha (\nabla f(w_k) + \lambda w_k).$$

En particulier, en partant de $w_0 = \bar{w}$:

$$w_1^{GD} = \bar{w} - \alpha (\nabla f(\bar{w}) + \lambda \bar{w}).$$

Or, sur le toy problem on a montré que $\nabla f(\bar{w}) = 0$. Donc :

$$w_1^{GD} = \bar{w} - \alpha \lambda \bar{w} = (1 - \alpha \lambda) \bar{w}.$$

Ainsi, GD contracte directement w_0 d'un facteur $(1 - \alpha \lambda)$.

Proximal gradient (PG) sur $f_2 = f + g$. Le proximal gradient applique :

$$v_k = w_k - \alpha \nabla f(w_k), \quad w_{k+1} = \text{prox}_{\alpha g}(v_k),$$

où $g(w) = \frac{\lambda}{2} \|w\|^2$. On utilise le proximal connu pour la régularisation ℓ_2 :

$$\text{prox}_{\alpha g}(v) = \arg \min_w \left\{ \frac{1}{2} \|w - v\|^2 + \alpha \frac{\lambda}{2} \|w\|^2 \right\} = \frac{1}{1 + \alpha \lambda} v.$$

En partant de $w_0 = \bar{w}$:

$$v_0 = \bar{w} - \alpha \nabla f(\bar{w}) = \bar{w},$$

car $\nabla f(\bar{w}) = 0$. Donc :

$$w_1^{PG} = \frac{1}{1 + \alpha \lambda} \bar{w}.$$

Comparaison de w_1^{GD} et w_1^{PG} . On obtient donc :

$$w_1^{GD} = (1 - \alpha \lambda) \bar{w}, \quad w_1^{PG} = \frac{1}{1 + \alpha \lambda} \bar{w}.$$

Les deux méthodes contractent $\bar{w}$, mais de manière différente :

- GD utilise une approximation “linéaire” du terme λw dans la mise à jour ;
- PG applique une contraction *exacte* via l’opérateur proximal, qui est toujours bien définie et plus stable numériquement.

Cette différence explique souvent la meilleure stabilité (et parfois la meilleure vitesse) du proximal gradient lorsqu’un terme de régularisation est présent.

8 Question 7 – Analyse des performances : GD vs Proximal Gradient

Dans cette partie, on compare numériquement deux méthodes pour minimiser la fonction régularisée

$$f_2(w) = f(w) + \frac{\lambda}{2} \|w\|^2,$$

où f est l’objectif non convexe (perte biweight) et $\lambda > 0$. Les méthodes comparées sont :

- **Gradient Descent (GD)** appliqué directement à f_2 ,

$$w_{k+1} = w_k - \alpha (\nabla f(w_k) + \lambda w_k),$$

- **Proximal Gradient (PG)** appliqué à la décomposition $f_2 = f + g$ avec $g(w) = \frac{\lambda}{2} \|w\|^2$,

$$v_k = w_k - \alpha \nabla f(w_k), \quad w_{k+1} = \text{prox}_{\alpha g}(v_k) = \frac{1}{1 + \alpha \lambda} v_k.$$

On analyse les courbes de convergence (objectif en échelle log) obtenues dans le notebook.

(a) D’après le plot, Proximal Gradient est-il meilleur que Gradient Descent ? Pourquoi ?

Observation sur les résultats. Sur la figure de convergence, les deux méthodes diminuent l’objectif, mais la courbe du proximal gradient est légèrement en dessous de celle de GD (ou décroît plus vite selon les réglages), ce qui indique une meilleure efficacité en pratique (meilleure valeur atteinte pour un même nombre d’itérations / de coût).

Explication. La différence s'explique par la manière de traiter la régularisation ℓ_2 :

- GD traite la régularisation via son gradient λw , c'est-à-dire de manière *explicite*. Cette mise à jour dépend fortement du choix du pas α et peut devenir trop agressive ou trop conservatrice si α n'est pas bien ajusté.
- PG traite la régularisation *exactement* via l'opérateur proximal :

$$\text{prox}_{\alpha g}(v) = \frac{1}{1 + \alpha\lambda}v,$$

ce qui correspond à une contraction contrôlée (*shrinkage*) toujours stable.

En particulier, en présence de régularisation, le proximal gradient peut être vu comme un schéma qui combine :

1. une descente sur la partie non convexe f ,
2. une correction exacte qui applique immédiatement l'effet de la pénalisation ℓ_2 .

Cela améliore souvent la stabilité et peut accélérer la convergence, ce qui est cohérent avec les courbes observées.

Conclusion. D'après le plot, **le proximal gradient est au moins aussi bon, et généralement meilleur que GD** sur ce problème régularisé, car il traite la régularisation ℓ_2 de manière exacte et stable.

(b) Que se passe-t-il lorsque λ devient beaucoup plus grand ? Est-ce cohérent avec les propriétés de la régularisation ℓ_2 ?

Observation attendue. Lorsque λ augmente fortement, le terme de régularisation

$$\frac{\lambda}{2}\|w\|^2$$

devient dominant dans l'objectif. L'optimisation cherche alors principalement à réduire $\|w\|$, ce qui pousse la solution vers le vecteur nul :

$$w^* \approx 0.$$

Numériquement, on observe typiquement :

- une contraction beaucoup plus forte des paramètres,
- une convergence rapide vers une solution de petite norme,
- un objectif régularisé influencé fortement par le terme $\frac{\lambda}{2}\|w\|^2$.

Interprétation (propriétés de ℓ_2). Ces observations sont cohérentes avec l'effet bien connu de la régularisation ℓ_2 :

- elle pénalise les grands coefficients et favorise des solutions de plus petite norme ;
- elle réduit la variance du modèle et le rend plus stable ;
- si λ est trop grand, le modèle est sur-régularisé et devient trop "simple" (solution proche de zéro), ce qui peut dégrader l'ajustement aux données.

Conclusion. Augmenter λ pousse les poids vers zéro et renforce l'effet de *shrinkage*. C'est exactement le comportement attendu d'une régularisation ℓ_2 et cela confirme la cohérence des résultats expérimentaux.

9 Conclusion

Dans ce projet, nous avons étudié un problème de régression robuste basé sur la fonction de perte biweight, qui permet de limiter l'influence des valeurs aberrantes au prix d'une perte de convexité. Ce cadre met en évidence plusieurs difficultés et phénomènes caractéristiques de l'optimisation non convexe.

Dans un premier temps, l'analyse du toy problem a permis d'illustrer concrètement la notion de *point selle*. Nous avons montré qu'un point critique peut ne pas être un minimum local, et que la descente de gradient peut s'y bloquer si l'on part d'un point initial particulier. Cependant, les expériences numériques confirment également un résultat théorique important : pour presque tout point initial, la descente de gradient échappe aux points selles et converge vers un minimum.

Dans un second temps, nous avons étudié le stochastic gradient. Les résultats montrent que, sur des jeux de données de grande taille, les méthodes stochastiques peuvent être plus efficaces que la descente de gradient classique, en particulier grâce à des mises à jour plus fréquentes. Nous avons également observé l'influence cruciale de la taille du batch, avec l'existence d'un compromis entre rapidité (petits batchs) et stabilité (batchs plus grands).

Enfin, l'introduction d'une régularisation ℓ_2 a permis de modifier profondément la géométrie du problème. La régularisation supprime certaines directions instables et rend l'optimisation plus robuste. Dans ce cadre, le proximal gradient s'est révélé particulièrement efficace, car il traite la partie régulière de manière exacte via l'opérateur proximal, ce qui améliore la stabilité et accélère la convergence par rapport à la descente de gradient classique.

Globalement, ce projet met en évidence l'importance de combiner analyse théorique et expérimentation numérique pour comprendre le comportement réel des algorithmes d'optimisation. Il illustre également que, dans des problèmes modernes de machine learning, le choix de l'algorithme (GD, SGD, méthodes proximales), du pas, et de la régularisation est déterminant pour obtenir des performances satisfaisantes.